

BYOD: BUILD YOUR OWN DLM

PARAMETER-EFFICIENT CONVERSION FROM AUTOREGRESSIVE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Diffusion language models (DLMs) permit parallel token updates and flexible generation orders, but leading systems rely on costly pretraining or continued full-model training. We ask whether diffusion generation can instead be added to existing instruction-tuned autoregressive models using a single parameter-efficient conversion recipe. Build Your Own DLM (BYOD) replaces causal attention with bidirectional attention and trains only query/value LoRA adapters under a masked denoising objective, leaving 94.2–95.9% of parameters frozen. Without architecture-specific tuning, the same recipe converts Gemma 2 9B, Llama 3.1 8B, Qwen2.5 7B, and Ministral 8B, with each 25,000-update conversion processing at most 102.4M token positions on a single GPU. All four converted models learn to generate coherent responses from fully masked sequences. On a preliminary nine-task evaluation, the strongest converted model reaches 50.7% average accuracy, compared with 54.0% for LLaDA 8B Instruct in the same evaluation harness. Continued training does not improve aggregate downstream accuracy, but improves generation quality when the number of denoising network function evaluations (NFE) is smaller than the output length. Together, these results provide a practical route for researchers to build diffusion language models from existing autoregressive checkpoints using only parameter-efficient adaptation and single-GPU compute.

1 INTRODUCTION

Autoregressive (AR) language models generate a sequence from left to right. This method has proven extremely effective due to its simple learning target and effective scaling capabilities. However, it also has trade-offs: it fixes the generation order, limits the model to unidirectional attention, and predicts only one new token at a time. Diffusion language models (DLMs) instead begin with a corrupted sequence and can iteratively reconstruct multiple positions at a time using bidirectional context (Austin et al., 2021a; Sahoo et al., 2024; Shi et al., 2024). Their number of function evaluations (NFE) can be chosen independently of output length, and early low-probability predictions can be revised. However, training or adapting models for diffusive generation has so far been computationally intensive and therefore expensive.

Our aim is to provide a practical alternative to training a separate diffusion foundation model for every model architecture family, reusing knowledge embedded in existing AR models reduces the data, compute, and financial barriers to experimentation. This emphasis follows calls to treat computational efficiency and access as research outcomes alongside accuracy (Strubell et al., 2019). We also estimate the potential sustainability impact, while recognizing that compute volume alone is not a carbon measurement: emissions depend strongly on hardware, datacenter efficiency, location, and energy supply (Patterson et al., 2021). We report bounded incremental training scale and an explicitly assumption-based operational-emissions scenario, while avoiding claims about measured emissions.

We study this question with BYOD, a generalizable conversion and evaluation pipeline for four instruction-tuned decoder families: Gemma 2 9B, Llama 3.1 8B, Qwen2.5 7B, and Ministral 8B. We selected the latest text-only instruction checkpoint from each family with a comparable parameter count. Multimodal variants are not used to avoid modality-specific training as an additional

confounder. Every backbone receives the same bidirectional attention intervention, LoRA target modules, and masked-diffusion objective. This paired design separates conversion quality from the strength of the starting checkpoint. We directly compare our models against LLaDA 8B Instruct because its standard masked-diffusion objective, instruction-tuning setup, and overlapping benchmark suite make it the cleanest external control. We additionally report published DiffuLLaMA, Dream-Instruct, iLLaDA-Instruct, and Efficient-DLM values, but keep them protocol-separated: their additional attention, masking, scheduling, data, and inference techniques would confound a controlled comparison.

Our contributions are:

- a single, easily reproducible, architecture-agnostic recipe that converts four instruction-tuned AR model families to masked diffusion using only LoRA parameter updates on a single Google Colab GPU;
- a paired evaluation of each diffusion model and its frozen AR parent across general knowledge, reasoning, mathematics, and code benchmarks;
- a controlled analysis of base-model choice, continued training, denoising-step count, validation loss, and externally scored generation perplexity; and
- a transparent comparison between results produced by our common harness and the published values reported with the LLaDA evaluation protocol.

2 RELATED WORK

Discrete diffusion models corrupt and reconstruct categorical sequences rather than adding Gaussian noise. D3PM formalized categorical corruption with transition matrices, including an absorbing process in which tokens transition to a dedicated mask state (Austin et al., 2021a). MDLM and contemporaneous work showed that this masked formulation admits a simple noise-weighted cross-entropy objective, while RADD reparameterized the predictor in terms of conditional distributions over clean tokens (Sahoo et al., 2024; Shi et al., 2024; Ou et al., 2025). These approaches form the basis of masked diffusion language models (MDLMs). Uniform (or multinomial) diffusion, which instead corrupts tokens with samples from the ordinary vocabulary, is another option (Hoogeboom et al., 2021); we use the simpler absorbing-mask denoising objective employed by LLaDA.

The authors of LLaDA were the first to show that masked diffusion can scale to an 8B-parameter general-purpose language model (Nie et al., 2025). Although its architecture follows Llama 3.1 8B, LLaDA was pre-trained from scratch using 2.3T tokens and 130,000 GPU-hours. AR-to-diffusion conversion instead amortizes knowledge already acquired by an AR checkpoint. DiffuLLaMA established this route through 65B tokens of full-model continued training at 7B scale (Gong et al., 2025). Dream adopts DiffuLLaMA’s shifted-prediction initialization, continues Qwen2.5 for 580B tokens, and adds context-adaptive token-level noise rescheduling (Ye et al., 2025). Efficient-DLM combines 500B tokens of continued training with blockwise attention and position-dependent masking (Fu et al., 2025), whereas iLLaDA returns to from-scratch diffusion pretraining at substantially larger scale (Nie et al., 2026). Diffusion conversion has also been demonstrated at larger sparse scales by RND1 (30B parameters), LLaDA2.0 (16B and 100B), and DiffusionGemma (25.2B, using uniform diffusion) (Chandrasegaran et al., 2025; Bie et al., 2025; DiffusionGemma Team, 2026). These methods establish scalability but are not directly comparable to the dense 3–9B systems in Table 1; BYOD deliberately tests a smaller intervention consisting of fully bidirectional attention and Q/V-only LoRA on instruction data.

Hybrid and blockwise models offer a different compromise between bidirectionality and deployability. Block Diffusion is autoregressive across blocks but diffusive within each block, enabling variable-length generation and inter-block KV caching (Arriola et al., 2025). Fast-dLLM v2 converts Qwen2.5 models using full-parameter fine-tuning with approximately 1B tokens and combines blockwise complementary masking with hierarchical caching (Wu et al., 2025). Sequential Diffusion Language Models (SDLM) instead learn next-sequence prediction and dynamically finalize variable-length subsequences, reporting conversion with 2.3 billion tokens (Liu et al., 2025). These systems retain more left-to-right structure than our fully bidirectional answer block, making them especially relevant to speed, caching, and arbitrary-length generation but not suited as direct controls for our intervention.

Table 1: Incremental training scale and standardized resource estimates for approximately comparable dense 3–9B diffusion systems. “Full” updates the main model weights; LoRA updates diffusion adapters. GPU-hours marked [†] are estimated from published model/data scale; unmarked values are reported. Costs are current cloud-rental equivalents and CO₂e is an operational scenario estimate (Appendix D), not a measured footprint. NR: insufficient information.

System	Diffusion-stage data	Updated weights	Hardware	GPU-h	Cost (USD)	CO ₂ e
BYOD (ours)	≤102.4M tokens	LoRA	1×RTX Pro 6000	3.2	\$11	0.9 kg
DiffuLLaMA 7B	65B tokens	Full	64×GH200	1,917 [†]	\$9,951	0.65 t
LLaDA 8B	2.3T tokens	Full	H800	130,000	\$674,830	44.1 t
Dream 7B	580B tokens + SFT	Full	NR	17,105 [†]	\$88,791	5.81 t
iLLaDA 8B	12T tokens + SFT	Full	NR	404,449 [†]	\$2,099,494	137.3 t
Efficient-DLM 8B	500B tokens	Full	128×H100	16,852 [†]	\$87,479	5.72 t
Uno 8B	7B tokens	LoRA	64×H200	3,840	\$19,933	1.30 t
Fast-dLLM v2 7B	~1B tokens	Full	64×A100	768	\$3,987	149 kg
SDLM 3B	2.3B source tokens	Full	128×GPU (type NR)	NR	NR	NR

LAD first showed that LoRA can induce diffusion generation, using a hybrid objective that combines masking with token swaps, duplications, and span shifts (Kuiper et al., 2025; Hu et al., 2022). BYOD instead tests whether a simpler IID absorbing-mask objective can transfer reliably across model families. Other LoRA-based diffusion work addresses different goals: Uno learns diffusion-draft adapters while retaining the AR model as verifier, the dLLM framework uses LoRA to fine-tune already-diffusive checkpoints rather than for its Tiny-A2D conversion, and NaRA adapts existing diffusion models with noise-conditioned LoRA (Sahoo et al., 2026; Zhou et al., 2026; Wang et al., 2026). Thus, their LoRA components do not test whether low-rank updates alone can convert several large AR model families into standalone diffusion models.

Table 1 places the incremental training budgets of related studies side by side. For conversion methods, these budgets exclude the substantial earlier cost of training the AR parent; for from-scratch methods they include diffusion pretraining. DiffusionGemma is discussed above but omitted from this resource table because its 25.2B-total/3.8B-active sparse architecture falls outside the table’s dense 3–9B scope and its report provides neither an absolute conversion-token count nor training accelerator-hours (DiffusionGemma Team, 2026). Dollar and operational-emissions columns are rental-equivalent estimates under one documented pricing and energy scenario, not the authors’ actual bills or measured footprints; Appendix D gives the complete calculation and its limitations.

Our study has explicitly chosen to hold the adaptation mechanism as constant as the architectures permit, repeat it across four families, and benchmark each converted model against the exact AR checkpoint from which it originated. This paired comparison is essential because it allows us to compare our results directly with the prior capability of the parent model.

3 METHOD

3.1 MASKED-DIFFUSION TRAINING OBJECTIVE

The conversion objective teaches the adapted model to recover clean answer tokens from randomly masked versions of the same response while conditioning on an uncorrupted instruction. Equation 1 formalizes this reconstruction loss and its correction for the sampled noise level; the following subsections describe the bidirectional attention intervention and the restricted set of parameters used to optimize it.

Let p be an observed instruction prompt and $x_0 = (x_0^1, \dots, x_0^L)$ its clean response region, where L is the response-region length and x_0^i is its token at position i . For each example, we sample $t \sim \mathcal{U}[10^{-3}, 1]$ and independently replace each eligible response token with a single tokenizer-native mask representation with probability t . Denote the resulting sequence by x_t and the sampled mask positions by M_t . The model is trained with same-position cross-entropy,

$$\mathcal{L}(\theta) = -\mathbb{E}_{t, x_0, x_t} \left[\frac{1}{t|E|} \sum_{i \in M_t} \log p_{\phi, \theta}(x_0^i | p, x_t) \right], \quad (1)$$

Table 2: Converted parent checkpoints and audited trainable parameter counts. All trainable parameters are Q/V LoRA weights.

Parent checkpoint	Total parameters	Trainable	Trainable (%)
Gemma 2 9B IT	10.732B	572.5M	5.33
Llama 3.1 8B Instruct	8.466B	436.2M	5.15
Qwen2.5 7B Instruct	7.939B	323.0M	4.07
Ministral 8B Instruct 2410	8.511B	490.7M	5.77

where E is the set of eligible answer-and-padding positions, $|E|$ is its cardinality, $M_t \subseteq E$ is the subset masked at noise level t , and $p_{\phi, \theta}$ is the model’s conditional token distribution. The expectation is over training examples, sampled noise levels, and their resulting corruptions; ϕ denotes the frozen parent parameters and θ the LoRA parameters. Prompt and chat-template tokens are always visible and never enter the loss. Training corruption is resampled online; validation corruption is deterministic from the example index and seed.

The clean terminating EOS is included as a genuine answer token. To teach the fixed-width generator to emit termination beyond the answer, dynamic batch padding is represented by repeated EOS tokens; these positions are also eligible for masking and loss. Thus padding is part of the learned denoising target rather than conventional ignored padding. This design choice should be interpreted separately from the semantic answer-token objective and is ablated in **XXX: appendix/table after final ablation selection**.

3.2 BIDIRECTIONAL CONVERSION AND TRAINABLE PARAMETERS

We replace the causal mask with an all-zero four-dimensional additive attention mask, so every prompt, answer, and EOS-padding position can attend to every position. The architecture and output vocabulary remain unchanged. We attach LoRA adapters to every self-attention query and value projection, with rank $r = 1024$, scaling $\alpha = 1024$, and dropout zero. This high rank reflects the scale of the intervention: the adapters must learn a new conditional denoising process rather than a narrow downstream task. In LAD’s rank study, increasing rank from 512 to 1024 at 3B improved every reported benchmark and perplexity, whereas increasing from 1024 to 2048 at 8B gave no consistent further gain; our preliminary rank experiments showed the same pattern. We therefore chose 1024 as a compromise between conversion capacity and training footprint (Kuiper et al., 2025). Backbone weights, token embeddings, the language-model head, biases, and normalization layers are frozen. Each model reuses an existing single-token string from its tokenizer (MASK for Gemma, Llama, and Qwen; $<?>$ for Ministral), avoiding vocabulary expansion.

Table 2 reports the parameter audits saved by the training runs. The trainable fraction varies because the four attention implementations have different projection shapes and layer counts.

After training, we fold each LoRA branch into its parent projection, $W_{\text{merged}} = W + (\alpha/r)BA$, where W is the frozen base projection, A and B are the trained low-rank adapter matrices, r is their rank, and α is the LoRA scaling parameter. We then remove the adapter modules. The resulting standalone checkpoint retains bidirectional inference but has the base architecture’s parameter count: Llama decreases from 8.466B adapter-loaded parameters to 8.030B merged parameters (16.06 GB in BF16). Thus the model-size increase is temporary, although optimizer states and activations still add training-time memory.

3.3 TRAINING DATA

All main models use the same training dataset and seed-selected ordering. Examples are retokenized with each parent model’s native chat template and tokenizer. Only the current answer region is corrupted. The dataset builder groups sources into general instruction following, multiple-choice reasoning, mathematics, and code, deduplicates normalized prompts, and blocks exact normalized matches with represented benchmark validation/test prompts. **XXX: Archive the exact Hub revision and manifest used by these original runs, then insert the final category/source counts and long-answer filtering policy.**

The four primary models see the same dataset prefix. The continued Llama condition initializes from the primary adapter and uses the next non-overlapping segment of the stable shuffle. It resamples corruption masks and restarts AdamW and its learning-rate schedule; it is therefore continued adapter training, not a restoration of optimizer state. Exact stage sizes are consolidated in Appendix B.

3.4 OPTIMIZATION

We use AdamW with peak learning rate 10^{-5} and train with BF16 master weights plus Transformer Engine HYBRID FP8 execution. All reported conversions ran on a single NVIDIA RTX PRO 6000 Blackwell Server Edition in Google Colab, without multi-GPU or multi-node parallelism; the code, memory strategy, and adapter scope were explicitly designed around this constraint. Appendix B gives the complete optimization, checkpointing, validation, and reproducibility configuration.

3.5 DIFFUSION GENERATION

Generation begins from a fully masked answer block. At each reverse step the model predicts every masked position, after which the least-confident positions are retained for reconsideration according to the LLaDA fixed-transfer schedule; newly accepted positions become permanent context. The primary factual and code benchmarks use greedy token selection and low-confidence remasking. Open-ended evaluation instead samples at temperature 0.7 and delays acceptance of EOS/EOT predictions. Complete task-specific diffusion and AR budgets are reported separately from training details in Appendix C.

4 EVALUATION

4.1 PAIRED MODELS AND BASELINES

For Gemma, Llama, Qwen, and Ministral we compare the diffusion adapter with greedy generation from its unchanged AR parent. LLaDA 8B Instruct is evaluated as an external diffusion baseline. We report two explicitly separated LLaDA comparisons: (i) LLaDA run through our prompts, sampling code, subsets, and scorers; and (ii) values transcribed from the LLaDA paper/evaluation suite. The latter are contextual references rather than directly matched measurements.

4.2 BENCHMARKS AND SCORING

Our validation suite contains MMLU, MMLU-Pro, HellaSwag, ARC-Challenge, GSM8K, MATH-500, GPQA-Main, HumanEval, and sanitized MBPP (Hendrycks et al., 2021a; Wang et al., 2024; Zellers et al., 2019; Clark et al., 2018; Cobbe et al., 2021; Hendrycks et al., 2021b; Rein et al., 2023; Chen et al., 2021; Austin et al., 2021b). HellaSwag uses its labelled validation split, ARC-Challenge and MATH-500 use test, and GPQA-Main uses the release’s train-named evaluation split. Multiple-choice answers are extracted by label; GSM8K uses the final numeric answer; MATH uses normalized boxed answers; and HumanEval/MBPP execute tests in isolated local subprocesses.

For 30 fixed open-ended prompts, Microsoft Phi-4 scores each generated answer autoregressively. Our primary aggregate is token-weighted perplexity, $\exp(\sum_j \text{NLL}_j / \sum_j N_j)$, so each scored token contributes equally; per-answer means and medians are retained as supplementary diagnostics. These are external-reference fluency measures, not likelihoods of the diffusion model itself. This distinction is important: generative perplexity inherits the reference model’s preferences and does not penalize loss of sample diversity (Turok et al., 2026). We therefore also compute sliding model-token Distinct-1/2/3 after excluding special tokens. Higher Distinct- n means a larger fraction of unique n -grams.

Table 3: Preliminary exact-match/pass@1 accuracy in our matched harness, with 50 examples per populated cell. D: converted diffusion model; AR: unchanged parent. Llama-50k shares the Llama-25k AR parent. Among the common-harness diffusion columns, bold marks the best result and underline the best converted BYOD result (ties retained); the published LLaDA column is protocol-separated and excluded from highlighting. Parenthesized averages cover only the indicated number of populated tasks; an em dash denotes an evaluation not yet available.

Task	Gemma 25k		Llama			Qwen 25k		Ministral 25k		LLaDA	
	D	AR	D-25k	D-50k	AR	D	AR	D	AR	Ours	Published
ARC-C	.880	.920	.800	.780	.740	.860	.920	.820	.740	.880	.885
GPQA	.240	.260	.260	.260	.340	.300	.400	.240	–	.240	.333
GSM8K	.540	.840	<u>.620</u>	.440	.820	.500	.900	.600	.860	.660	.694
HellaSwag	.780	.680	.700	.780	.700	.740	.780	.820	.720	.700	.746
HumanEval	.380	.740	<u>.460</u>	.420	.480	.380	.960	–	–	.500	.494
MATH-500	<u>.300</u>	.640	.220	.140	.480	.280	.780	–	–	.440	.319
MBPP	.460	.740	.400	.480	.760	.500	.820	–	–	.480	.410
MMLU	.760	.720	.640	.700	.540	.580	.660	.560	.520	.560	.655
MMLU-Pro	.220	.500	.280	.280	.400	.280	.400	<u>.300</u>	.320	.400	.370
Average	<u>.507</u>	.671	.487	.476	.584	.491	.736	.557 (6)	.632 (5)	.540	.545

Table 4: Token-weighted Phi-4 perplexity on 128-token open-ended generations at 64 denoising steps. Lower is better, but this is a secondary fluency diagnostic.

Model family/checkpoint	Diffusion	AR parent
Gemma 2 9B, 25k	8.017	3.011
Llama 3.1 8B, 25k	10.503	2.489
Llama 3.1 8B, 50k	8.709	2.489
Qwen2.5 7B, 25k	10.272	2.230
Ministral 8B, 25k	9.424	2.305
LLaDA 8B Instruct	4.065	–

5 RESULTS

5.1 PRELIMINARY BENCHMARK ACCURACY

Table 3 reports the currently completed matched-harness evaluations. Empty cells indicate evaluations that are not yet available. Because each populated cell currently contains 50 examples, these preliminary values should not yet be used for significance claims.

The provisional complete diffusion macro averages are 50.7% for Gemma, 48.7% for Llama-25k, 47.6% for Llama-50k, 49.1% for Qwen, and 54.0% for LLaDA in our harness. Ministral’s 55.7% diffusion and 63.2% AR averages are explicitly partial. The matched AR parents outperform their converted counterparts on average, although individual tasks differ. Notably, the second Llama stage does not uniformly improve downstream accuracy despite reducing its 64-step generation perplexity (Section 5.2), motivating checkpoint-wise evaluation rather than selecting solely by denoising loss or fluency. Published values from other diffusion systems are provided separately in Appendix A because their protocols are not matched to ours.

5.2 OPEN-ENDED FLUENCY AND DENOISING BUDGET

Increasing the denoising budget strongly improves this diagnostic. For example, Llama-25k changes from 30.049 at 32 steps to 10.503 at 64 and 3.753 at 128; Gemma-25k changes from 21.795 to 8.017 and 3.317. We call settings with fewer denoising network evaluations than generated tokens ($NFE < L$) the *parallel decoding regime*; unlike AR decoding, they permit more than one token to be finalized per model evaluation. Perplexity values are meaningful only under matched output length, sampler, temperature, and denoising budget.

We also benchmarked the saved Llama adapter before and after merging (Appendix E). Merging removes 436.2M deployed parameters while changing token-weighted perplexity from 19.381 to

Table 5: Training progression: token-weighted Phi-4 perplexity for 128-token generations.

Model	Steps	1k	10k	20k	30k	40k	50k
Llama	32	15.502	23.138	17.091	14.737	17.201	16.797
Llama	64	10.256	10.015	9.646	9.839	8.716	8.382
Llama	128	5.795	5.287	4.763	4.664	4.420	4.876
Model	Steps	1k	5k	10k	15k	20k	25k
Gemma	32	19.206	13.235	11.547	12.413	10.398	11.688
Gemma	64	10.387	7.506	7.941	6.746	6.751	7.826
Gemma	128	5.852	4.457	4.313	3.454	3.941	3.872

19.283 (−0.51%) at 32 NFE and from 3.753 to 3.720 at 128 NFE; Distinct-1/2/3 at 32 NFE change from .495/.788/.899 to .498/.807/.918. On the preliminary nine-task subsets, the unweighted accuracy mean changes from 51.3% to 49.3%, with mixed task-level differences. Across these metric families, merging preserves the model’s overall behavior without a systematic open-generation penalty.

5.3 PROGRESSION WITH TRAINING UPDATES

Table 5 shows checkpoint-wise Phi-4 perplexity. Improvements are not monotonic at every budget. The matched final-checkpoint comparison suggests that the second Llama stage primarily helps in the parallel decoding regime: for 128-token outputs, 25k to 50k training lowers perplexity from 30.049 to 16.797 at 32 NFE and from 10.503 to 8.709 at 64 NFE, while the 128-NFE value worsens from 3.753 to 4.876. Meanwhile, the preliminary factual-task macro average changes from 48.7% to 47.6%. Continued training therefore appears to improve low-budget parallel decoding rather than general factual capability; ongoing evaluations will test whether this pattern holds across models and seeds. Gemma reaches most of its 64-step improvement within 15–20k updates and slightly degrades by 25k. Final benchmark results will report fixed checkpoints rather than choosing a separate best checkpoint per task.

5.4 BASE-MODEL DEPENDENCE AND LLADA COMPARISON

All four converted families produce usable diffusion generations, but the resulting quality is not architecture invariant. Gemma currently has the strongest complete BYOD diffusion macro average, while Qwen and Llama are similar on average but differ by task. The incomplete Ministral row already illustrates another important point: its strong multiple-choice and GSM8K scores do not establish code or MATH capability until those evaluations finish. **XXX: Add confidence intervals and formal paired comparisons once the evaluation size is expanded.**

LLaDA scores 54.0% on the nine-task unweighted average in our harness, compared with 54.5% from its published values. Agreement varies by task and should not be interpreted as an exact reproduction because prompts, subsets, generation lengths, answer extraction, and code execution may differ. We therefore present our common-harness comparison as primary and Appendix A only as external calibration. We chose LLaDA for direct external testing because it uses the uncomplicated response-masking objective closest to ours and reports the largest overlap with our task suite. Dream, iLLaDA, and Efficient-DLM add different data scales, noise weighting, SFT formats, scoring rules, attention patterns, or positional masking. Importing those techniques into the primary experiment would prevent us from attributing outcomes to parameter-efficient conversion itself.

6 DISCUSSION

The central empirical finding is qualitative but robust across the selected runs: high-rank attention-only LoRA is sufficient to turn four independently developed causal instruction models into generators that reconstruct answers from a fully masked block. This supports the view that much of the knowledge needed for diffusion generation is already present in AR representations and vocabulary heads. Because each conversion fits on one Google Colab GPU rather than requiring a dedicated multi-accelerator cluster, the method also lowers the infrastructure threshold for reproducing, modifying, and teaching large-scale diffusion-language-model conversion. A capable paid cloud GPU is still required, so this is broader access rather than zero-cost access.

The merge result also shows that LoRA’s parameter-count overhead is not permanent: the deployed Llama returns from 8.466B to 8.030B parameters without a material open-generation degradation in this small matched comparison (Appendix E). Algebraically folding $(\alpha/r)BA$ into W removes both the stored adapter branch and its extra low-rank inference matrix multiplications. Consequently, using hundreds of millions of high-rank LoRA parameters during conversion need not make the final generator larger or more expensive per network evaluation than the base architecture. This is not model compression—the full base-sized checkpoint remains—and diffusion decoding can still require many network evaluations; more prompts, deterministic parity checks, and backbones are needed to bound small numerical or sampling differences after merging.

Conversion is not free, however. First, 323–573M trainable parameters are parameter-efficient relative to full-model training but substantially larger than conventional low-rank task adapters. Second, our secondary perplexity remains worse than each AR parent and than LLaDA at 64 steps. Third, additional Llama training does not improve the provisional downstream benchmark aggregate, but its largest fluency gains occur when $NFE < L$. This suggests a more specific benefit: additional denoising training may teach the model to resolve multiple mutually dependent positions per evaluation, improving the quality–compute trade-off without adding corresponding benchmark capability. The 128-NFE result cautions that this interpretation remains preliminary. Finally, EOS-padding supervision makes the fixed-width objective different from a pure answer-only likelihood and deserves explicit ablation.

Perplexity alone is insufficient for selecting a diffusion generator. Our soft-frontier masking ablation lowered Phi-4 generative perplexity but substantially reduced sliding model-token Distinct-1/2/3 and produced visibly more repetitive answers (Appendix F). DiffuLLaMA similarly pairs external-model perplexity with distinct bigrams to check that low perplexity was not obtained through repetition (Gong et al., 2025). More fundamentally, external generative perplexity reflects a separate AR model’s preferences and ignores diversity, so repeated high-probability text can score deceptively well (Turok et al., 2026). Theory likewise distinguishes token-level perplexity from whole-sequence correctness: a masked diffusion model may approach favorable perplexity with few steps while still requiring a step count that scales with sequence length for low sequence error (Feng et al., 2025). We therefore triangulate external perplexity with Distinct-1/2/3, task accuracy, and generated samples. Distinct- n is itself only a repetition diagnostic, so grammaticality, coherence, and actual usability ultimately require systematic qualitative inspection rather than replacement by any single automatic score.

The literature comparison sharpens the trade-off. DiffuLLaMA and Dream reuse AR initialization but update the full model for 65B and 580B tokens, respectively; Efficient-DLM invests 500B tokens and architectural training choices specifically designed to retain causal-model behavior and enable cached block decoding (Gong et al., 2025; Ye et al., 2025; Fu et al., 2025). iLLaDA’s strong reported results come from 12T-token diffusion pretraining plus repeated large-scale SFT, a fundamentally different compute regime (Nie et al., 2026). Fast-dLLM v2 and SDLM show that retaining blockwise or sequential structure can reduce conversion cost while enabling practical caching and variable-length generation (Wu et al., 2025; Liu et al., 2025); our complementary question is how far a fully bidirectional model can be converted with adapter-only updates. Uno is closer to our parameter-efficient philosophy because it also trains LoRA diffusion weights, but its goal is different: an AR verifier guarantees samples from the original AR distribution, whereas BYOD seeks an independently usable fully bidirectional generator (Sahoo et al., 2026). Thus the higher published scores in Appendix A do not isolate a single superior masking objective; they bundle substantially different data budgets and modeling or decoding machinery.

The incremental scale is central to the intended contribution. One primary BYOD conversion processes orders of magnitude fewer token positions than the full-model conversion systems in Table 1, while leaving most parent parameters frozen. This comparison does not erase the sunk cost of AR pretraining, nor prove a proportional reduction in energy or emissions. It does show that an existing foundation checkpoint can be repurposed without repeating that pretraining or reserving a large cluster. That distinction matters both economically and for widening participation in diffusion-model research; direct power and carbon measurements remain necessary for an environmental comparison (Strubell et al., 2019; Patterson et al., 2021).

This is also why we do not claim that Appendix A is a leaderboard. Our direct LLaDA run holds prompts, subsets, extraction, and generation code fixed. The remaining columns are literature context,

useful for locating the attainable performance range but unsuitable for causal comparison. A future study can add these systems to the shared harness one at a time, provided their task-specific inference procedures are faithfully reproduced—a requirement emphasized by the dLLM framework (Zhou et al., 2026).

7 LIMITATIONS

The current benchmark results use 50 examples per reported task and do not yet contain every Ministral evaluation. Unless expanded, these estimates have high sampling uncertainty. Only one training seed is available per model, so architecture and random-seed effects are not separated. The four backbones differ in tokenizer, pretraining data, alignment, attention implementation, and nominal parameter count. Although chosen as the latest comparable text-only releases when the study was designed, they are not guaranteed to remain the latest checkpoints at publication time. Phi-4 perplexity favors text that its own AR distribution considers probable, ignores diversity, and is not a diffusion likelihood. Distinct- n detects local repetition but not factuality, grammar, or global coherence. Published LLaDA numbers are not directly comparable to our harness. We also have not yet established wall-clock speedups: $NFE < L$ reduces sequential model calls but does not automatically imply lower latency because each call processes the full answer block. Finally, we did not instrument the runs for energy consumption or carbon intensity. Single-GPU execution and lower incremental training volume support accessibility and compute-efficiency claims, not a quantified sustainability claim.

8 CONCLUSION

BYOD provides a common, parameter-efficient route from instruction-tuned AR checkpoints to masked diffusion generators across Gemma, Llama, Qwen, and Ministral. The preliminary evidence shows broad transfer of the conversion mechanism, but also substantial dependence on the parent model, inference budget, and evaluation metric. By fitting every conversion on a single Google Colab GPU, the work offers a reproducible pathway for researchers who cannot retrain a foundation model or provision a large accelerator cluster. Continued training appears more promising for improving generation in the parallel decoding regime than for improving the aggregate downstream benchmark score, a hypothesis the final checkpoint sweep will test. Completing the paired AR evaluation will determine not merely whether these models can diffuse, but how much of their original capability survives conversion and whether that modest incremental cost is justified relative to purpose-trained diffusion models.

REPRODUCIBILITY STATEMENT

Training and evaluation code is available at <https://github.com/RuurdKuiper/lad-g-eneric>. Public model cards and interactive demos for all four conversions are available under <https://huggingface.co/Ruurd>; each model card links to its corresponding full-precision ZeroGPU demo. The main text specifies the objective, trainable modules, data volume, optimization, generation, benchmarks, and scoring. The accompanying artifacts include resolved configurations, parameter audits, checkpoint identifiers, loss logs, generated samples, benchmark manifests, and frontier-ablation outputs; immutable dataset and software revisions will be recorded with the archival release.

ETHICS STATEMENT

This study uses existing language models and public benchmark or instruction datasets and does not involve human subjects. Converted models inherit factual errors, biases, unsafe behaviors, licensing constraints, and memorization risks from their parent checkpoints and training data. We report aggregate task performance rather than claiming safety or deployment readiness; release artifacts should preserve upstream licenses and document the data provenance and intended research use.

AI USE STATEMENT

Generative AI tools were used to assist software development and debugging, design and review of experiments, creation of analysis scripts, interpretation of preliminary results, literature discovery and summarization, and drafting and editing this manuscript. The authors manually reviewed AI-assisted code and prose, checked numerical claims against saved experimental artifacts, and verified literature claims and citations against primary sources. No benchmark value was entered solely from an unsupported model response. The authors take responsibility for the final content, claims, code, and artifacts.

REFERENCES

- Amazon Web Services. Amazon EC2 capacity blocks for ML pricing. <https://aws.amazon.com/ec2/capacityblocks/pricing/>, 2026a. Accessed 2026-09-21; reports \$5.191 per accelerator-hour for p5.4xlarge in US East.
- Amazon Web Services. How synthesia optimizes generative AI video inference on amazon EC2 G7e instances. <https://aws.amazon.com/blogs/architecture/how-synthesia-optimizes-generative-ai-video-inference-on-amazon-ec2-g7e-instances/>, 2026b. Accessed 2026-09-21; reports \$3.36 per hour for g7e.2xlarge in US East (Ohio).
- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.
- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, 2021a.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021b.
- Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, Chengxi Li, Chongxuan Li, Jianguo Li, Zehuan Li, Huabin Liu, Ling Liu, Guoshan Lu, Xiaocheng Lu, Yuxin Ma, Jianfeng Tan, Lanning Wei, Ji-Rong Wen, Yipeng Xing, Xiaolu Zhang, Junbo Zhao, Da Zheng, Jun Zhou, Junlin Zhou, Zhanchao Zhou, Liwang Zhu, and Yihong Zhuang. LLaDA2.0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.
- Keshigeyan Chandrasegaran, Armin W. Thomas, Jerome Ku, Federico Berto, Jae Myung Kim, Garyk Brixi, Eric Nguyen, Stefano Massaroli, and Michael Poli. RND1: Simple, scalable AR-to-diffusion conversion. Radical Numerics technical report, October 2025. URL https://www.radicalnumerics.ai/assets/rnd1_report.pdf.
- Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. In *arXiv preprint arXiv:1803.05457*, 2018.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Rei Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- DiffusionGemma Team. DiffusionGemma technical report. *arXiv preprint arXiv:2608.00146*, 2026.
- Guhao Feng, Yihan Geng, Jian Guan, Wei Wu, Liwei Wang, and Di He. Theoretical benefit and limitation of diffusion language model. *arXiv preprint arXiv:2502.09622*, 2025.

- Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, Maksim Khadkevich, Jan Kautz, Yingyan Celine Lin, and Pavlo Molchanov. Efficient-DLM: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2025.
- Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Jiawei Han, Hao Peng, and Lingpeng Kong. Scaling diffusion language models via adaptation from autoregressive models. In *International Conference on Learning Representations*, 2025.
- Google Cloud. Building software sustainably. <https://cloud.google.com/blog/topics/sustainability/building-software-sustainably>, 2025. Accessed 2026-09-21; reports fleet-wide PUE of 1.09.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021a.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems*, 2021b.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 30016–30030, 2022.
- Emiel Hoogeboom, Didrik Nielsen, Priyank Jaini, Patrick Forré, and Max Welling. Argmax flows and multinomial diffusion: Learning categorical distributions. In *Advances in Neural Information Processing Systems*, volume 34, pp. 12454–12465, 2021.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- International Energy Agency. Electricity 2025: Emissions. <https://www.iea.org/reports/electricity-2025/emissions>, 2025. Reports global electricity-generation intensity of 445 g CO₂/kWh in 2024.
- Ruurd J. A. Kuiper, Lars de Groot, Bram van Es, Maarten van Smeden, and Ayoub Bagheri. LAD: LoRA-adapted diffusion. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 97–110, 2025.
- Loïc Lannelongue, Jason Grealey, and Michael Inouye. Green algorithms: Quantifying the carbon footprint of computation. *Advanced Science*, 8(12):2100707, 2021. doi: 10.1002/advs.202100707.
- Yangzhou Liu, Yue Cao, Hao Li, Gen Luo, Zhe Chen, Weiyun Wang, Xiaobo Liang, Biqing Qi, Lijun Wu, Changyao Tian, Yanting Zhang, Yuqiang Li, Tong Lu, Yu Qiao, Jifeng Dai, and Wenhai Wang. Sequential diffusion language models. *arXiv preprint arXiv:2509.24007*, 2025.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Shen Nie, Qiyang Min, Shaoxuan Xu, Zihao Huang, Yuxuan Song, Yong Shan, Yankai Lin, Wayne Xin Zhao, Chongxuan Li, and Ji-Rong Wen. Improved large language diffusion models. *arXiv preprint arXiv:2606.25331*, 2026.
- NVIDIA. NVIDIA A100 Tensor Core GPU data sheet. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>, 2021. Accessed 2026-09-21; SXM maximum thermal design power 400 W.

- NVIDIA. NVIDIA H100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/h100/>, 2023. Accessed 2026-09-21; dense BF16 Tensor Core peak 989 TFLOP/s and configurable TDP up to 700 W.
- NVIDIA. NVIDIA RTX PRO 6000 Blackwell Server Edition. <https://www.nvidia.com/en-us/data-center/rtx-pro-6000-blackwell-server-edition/>, 2025. Accessed 2026-09-21; maximum power consumption up to 600 W.
- Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *International Conference on Learning Representations*, 2025.
- David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.
- Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, Ruihang Li, Miaosen Zhang, Chen Li, Jia Ning, Ruizhe Wang, Zheng Zhang, Shuguang Liu, Joe Chau, Han Hu, and Peng Cheng. FP8-LM: Training FP8 large language models. *arXiv preprint arXiv:2310.18313*, 2023.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.
- Subham Sekhar Sahoo, Lingjie Chen, Khiem Pham, Jonathan Geuter, Chaitanya Dwivedi, Varad Pimpalkhute, Yash Akhauri, Alexander Moreno, Mikhail Yurochkin, Zhenting Wang, Mostafa Elhoushi, Nolan Dey, Shane Bergsma, Joel Hestness, John Thickstun, Eric Xing, and Zhengzhong Liu. Unlocking lossless speedups in LLMs via discrete diffusion. *arXiv preprint arXiv:2609.04010*, 2026.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. *arXiv preprint arXiv:2406.04329*, 2024.
- Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650, 2019. doi: 10.18653/v1/P19-1355.
- Gilad Turok, Chris De Sa, and Volodymyr Kuleshov. DUEL: Exact likelihood for masked diffusion via deterministic unmasking. *arXiv preprint arXiv:2603.01367*, 2026.
- Shuaidi Wang, Zhan Zhuang, Ruping Huang, and Yu Zhang. NaRA: Noise-aware LoRA for parameter-efficient fine-tuning of diffusion LLMs. *arXiv preprint arXiv:2605.29716*, 2026.
- Yubo Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dLLM v2: Efficient block-diffusion LLM. *arXiv preprint arXiv:2509.26328*, 2025.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of ACL*, 2019.
- Zhanhui Zhou, Lingjie Chen, Hanghang Tong, and Dawn Song. dLLM: Simple diffusion language modeling. *arXiv preprint arXiv:2602.22661*, 2026.

A PROTOCOL-SEPARATED LITERATURE RESULTS

Table 6 places our primary Llama conversion beside values reported by other diffusion-model papers (Nie et al., 2025; Ye et al., 2025; Nie et al., 2026; Fu et al., 2025). The BYOD column uses our 50-example common harness, whereas every other column uses its source paper’s evaluation protocol. Differences in prompts, subsets, generation budgets, answer extraction, and execution environments prevent direct ranking; the table is included only to provide scale and task coverage. NR means that the source did not report a value for that task.

Table 6: Our Llama result and protocol-separated published diffusion-model accuracies. Bold marks the largest reported value in each row; it does not imply a controlled comparison. No cross-task averages are shown because task coverage differs by model.

Task	BYOD-Llama (ours)	LLaDA	Dream-Inst.	iLLaDA-Inst.	Efficient-DLM 8B
ARC-C	.800	.885	NR	NR	.619
GPQA	.260	.333	.330	NR	NR
GSM8K	.620	.694	.810	.890	.883
HellaSwag	.700	.746	NR	NR	.725
HumanEval	.460	.494	.555	.659	.683
MATH	.220	.319	.392	.567	.501
MBPP	.400	.410	.588	.580	.610
MMLU	.640	.655	.670	.716	.772
MMLU-Pro	.280	.370	.433	.523	NR

B COMPLETE TRAINING CONFIGURATION

Table 7 records the common resolved configuration of the four primary conversions. The continued Llama condition repeats the same stage from the first adapter checkpoint on the next non-overlapping data segment, with newly sampled masks and a freshly initialized optimizer and schedule; its checkpoints are spaced every 5,000 updates.

Table 7: Full common training configuration. Model-specific parent checkpoints, mask strings, and parameter counts appear in Table 2.

Setting	Value	Setting	Value
Examples per stage	400,000	Optimizer updates per stage	25,000
Maximum sequence length	256	Dynamic padding multiple	16
Batch size	16/16	Gradient accumulation	1
Optimizer	AdamW	Peak learning rate	10^{-5}
Schedule	cosine	Warmup	100 updates
Weight decay	0.01	Maximum gradient norm	0.5
Training precision	FP8 HYBRID	Master/evaluation precision	BF16
Gradient checkpointing	disabled	Selected-logit optimization	enabled
Corruption	IID mask-only	Noise time	$t \sim \mathcal{U}[0.001, 1]$
Mask-eligible region	answer + EOS padding	Loss region	corrupted answer + EOS padding
Answer/padding loss weights	0.9/0.1	Genuine answer EOS	included
LoRA targets	query/value projections	LoRA rank/scaling/dropout	1024/1024/0
Trainable normalization	disabled	Quantization	none
Validation	200 examples/250 updates	Checkpoint interval	1,000 updates
Data-loader workers	4	Prefetch factor	4

C COMPLETE INFERENCE CONFIGURATION

Table 8 separates task-specific diffusion budgets from the training configuration. Unless overridden in the table, diffusion evaluation uses the official LLaDA-style fixed-transfer sampler, greedy predictions (temperature 0), low-confidence remasking, classifier-free guidance scale 0, one full-answer block, and no repetition penalty. Delayed EOS/EOT means that a predicted terminator is assigned the lowest retention confidence and may therefore be predicted but is accepted late; HumanEval instead

Table 8: Task-specific masked-diffusion inference settings in the validation notebook. NFE is the number of denoising network evaluations.

Task	Output tokens	NFE	Block length	Temperature	Terminator handling
Open-ended	128	128/64/32	128	0.7	delayed retention
MMLU	64	64	64	0	ordinary
MMLU-Pro	128	128	128	0	delayed retention
HellaSwag	64	64	64	0	ordinary
ARC-Challenge	64	64	64	0	ordinary
GSM8K	256	256	256	0	delayed retention
MATH-500	256	256	256	0	delayed retention
GPQA-Main	128	128	128	0	delayed retention
HumanEval	256	256	256	0	delayed retention
MBPP	128	128	128	0	delayed retention

suppresses terminator logits. The open-ended diagnostic uses temperature 0.7 and 30 fixed prompts. Archived run manifests take precedence over mutable notebook defaults.

Autoregressive parents are decoded greedily. Their maximum-new-token budgets are reported separately because an AR token budget is not an NFE budget and should not be read as computational equivalence.

Table 9: Maximum new tokens for paired greedy AR evaluation.

Task	Tokens	Task	Tokens
Open-ended	128	MMLU	64
MMLU-Pro	2,048	HellaSwag	64
ARC-Challenge	64	GSM8K	256
MATH-500	256	GPQA-Main	128
HumanEval	256	MBPP	128

D COST AND OPERATIONAL-EMISSIONS ESTIMATION

Table 1 compares the cost of training a model that produces diffusion behavior. It should thus be taken into account that some of these models build on an AR parent, whereas others are trained from scratch. It also excludes exploratory runs, failed jobs, dataset construction, evaluation, storage, and inference. The estimates must therefore not be interpreted as total project or lifecycle footprints.

For BYOD we use the observed 3.2 GPU-hours for the conversion of a Llama 3.1 8B model for diffusive generation. For a prior work that reports GPU-hours, we retain that value. Where a paper reports model size but not time, we approximate training compute as $C \approx 6ND$ FLOPs (Hoffmann et al., 2022), where C is total training compute, N is the number of model parameters, and D is the number of processed training-token positions. We then calculate

$$G_{\text{est}} = \frac{6ND}{0.40 (989 \times 10^{12}) 3600}, \quad (2)$$

where G_{est} is aggregate estimated GPU-hours, 0.40 is the assumed model FLOPs utilization (MFU), 989×10^{12} is the NVIDIA H100’s dense BF16 Tensor Core throughput in FLOP/s (NVIDIA, 2023), and 3600 converts seconds to hours. The 40% MFU assumption is consistent with the 39.0–45.0% BF16 MFUs reported by FP8-LM for GPT-style 7B, 13B, and 175B training on H100 GPUs (Peng et al., 2023). Actual utilization varies considerably with architecture, sequence length, parallelization, communication, activation recomputation, and implementation efficiency. The assumption is therefore a standardized order-of-magnitude convention rather than a reconstruction of any authors’ cluster. It yields the daggered values in Table 1. The Dream and iLLaDA estimates cover their main 580B- and 12T-token stages but omit the smaller instruction-tuning additions, so they are lower bounds.

Table 10 calibrates the same calculation against studies that report both a usable token count and elapsed accelerator-hours (Nie et al., 2025; Wu et al., 2025). For LLaDA, it predicts approximately

77,519 GPU-hours, compared with 130,000 reported H800 GPU-hours, corresponding to an implied utilization of 23.9%. Thus 40% can be optimistic about achieved utilization and correspondingly conservative with respect to estimated accelerator-hours, rental cost, and emissions. Fast-dLLM v2 displays a still larger gap. Its specialized training concatenates clean and noised views and uses a nonstandard attention pattern, illustrating that an “implied MFU” from $6ND$ is a calibration diagnostic, not a measured hardware-utilization statistic.

Table 10: Calibration of the $6ND$ estimator against reported accelerator-hours. Predictions use 40% MFU and the stated accelerator’s dense-BF16 peak (989 TFLOP/s H100-class proxy for H800; 312 TFLOP/s for A100) (NVIDIA, 2023; 2021). “Implied MFU” is the utilization that would reconcile $6ND$ with reported time; it is not a measured MFU.

System	Parameters	Tokens	Pred. GPU-h	Reported GPU-h	Implied MFU
LLaDA	8B	2.30T	77,519	130,000	23.9%
Fast-dLLM v2	7B	1.31B	123	768	6.4%

All values in Table 1 and Table 10 are reproduced by the supplementary script `scripts/estimate_paper_compute.py`; its machine-readable output is included alongside the paper artifacts.

We do not apply this estimator to SDLM. Its released 3B training script launches 128 GPU workers, but neither accelerator type nor elapsed training time is reported (Liu et al., 2025). More importantly, SDLM’s 2.3B figure describes source-corpus tokens: its parallel-block construction rearranges each example and may append multiple noise/target blocks within a 5,632-token training sequence. Consequently, substituting 2.3B directly for D would undercount the token positions actually processed, while the $6ND$ approximation would also omit the material sequence-length-dependent attention cost. We therefore report its known GPU count but leave GPU-hours, cost, and emissions as NR rather than present a misleading lower-bound estimate.

We convert BYOD time to a rental-equivalent cost using the September 2026 AWS US-East on-demand price of \$3.36 per hour for a one-GPU `g7e.2xlarge`, which contains the same 96-GB NVIDIA RTX PRO Server 6000 accelerator used in our Colab run (Amazon Web Services, 2026b). For other systems we use one consistent current major-cloud proxy: \$5.191 per accelerator-hour for an AWS `p5.4xlarge` H100 Capacity Block in US East (Amazon Web Services, 2026a). Thus these figures answer “what would the stated or estimated accelerator-hours cost at a public contemporary rate?”, not what the original authors paid. They exclude storage, egress, idle reservation time, taxes, and discounts.

Operational emissions are estimated as energy use multiplied by the data center’s power usage effectiveness (PUE) and the grid carbon intensity (Lannelongue et al., 2021; Patterson et al., 2021):

$$\hat{E}_{\text{CO}_2\text{e}} = G P_{\text{GPU}} \text{PUE} I_{\text{grid}}. \quad (3)$$

Here $\hat{E}_{\text{CO}_2\text{e}}$ is estimated operational emissions in kg CO_2e , G is aggregate GPU-hours, P_{GPU} is assumed per-GPU power draw in kW, PUE is the dimensionless data-center power usage effectiveness, and I_{grid} is grid carbon intensity in kg $\text{CO}_2\text{e}/\text{kWh}$. We use 0.60 kW for RTX PRO 6000 runs, matching NVIDIA’s stated maximum board power (NVIDIA, 2025), and a 0.70-kW H100-class proxy for all other entries except the reported A100 run, for which we use 0.40 kW (NVIDIA, 2023; 2021). We assume PUE 1.09, Google’s reported fleet-wide value (Google Cloud, 2025), and the IEA’s 2024 global-average generation intensity of 0.445 kg $\text{CO}_2\text{e}/\text{kWh}$ (International Energy Agency, 2025). For example, one BYOD stage gives $3.2 \times 0.60 \times 1.09 \times 0.445 = 0.93$ kg CO_2e . These are location-independent GPU-only operating scenarios: they assume full rated GPU power, omit host CPU/memory/network energy and embodied hardware emissions, and do not account for renewable-energy procurement. Direct power telemetry and the datacenter’s time- and location-specific marginal carbon intensity would be required for a measured footprint.

E ADAPTER-MERGING ABLATION

Merging is an algebraic reparameterization rather than an additional training stage: each base projection W is replaced by $W + (\alpha/r)BA$, where A and B are the trained rank- r LoRA matrices

Table 11: Open-generation comparison of the adapter-loaded and merged Llama checkpoints. PPL is token-weighted; Distinct- n is averaged over answers. Both use 128 generated tokens; D-1/2/3 use 32 NFE.

Representation	Parameters	PPL ₃₂ ↓	PPL ₁₂₈ ↓	D-1↑	D-2↑	D-3↑
Adapter loaded	8.466B	19.381	3.753	.495	.788	.899
Merged standalone	8.030B	19.283	3.720	.498	.807	.918

Table 12: Factual benchmark accuracy (%) before and after merging. These are preliminary 50-example task subsets; the mean is unweighted across the nine tasks.

Task	Adapter	Merged	Task	Adapter	Merged
ARC-C	82.8	82.0	MATH	22.0	22.0
GPQA	28.0	30.0	MBPP	46.0	44.0
GSM8K	65.2	58.0	MMLU	61.2	64.0
HellaSwag	73.2	68.0	MMLU-Pro	33.2	28.0
HumanEval	50.0	48.0	Mean	51.3	49.3

and α is their scaling parameter. Subject to floating-point rounding, evaluating the folded projection is equivalent to evaluating the base and LoRA branches separately. The open-generation metrics are slightly better after merging, whereas the small factual subsets show mixed task differences and a 2.0-point lower macro mean. These reruns therefore support preservation of overall behavior, but are not precise enough to attribute task-level changes to merging; a logit-level deterministic parity test would isolate numerical merge error from generation and subset variance.

F FRONTIER-MASKING ABLATION

The selected main runs use the simpler IID corruption process: after sampling the example-level noise fraction t , each eligible answer or EOS-padding position is independently masked with probability t . In a later ablation, we tested *soft-frontier masking*, intended to resemble partially completed left-to-right text while retaining stochastic exceptions. For a frontier example, masking probability increases sigmoidally with answer position around a frontier determined by t ; the tested configuration used a 0.75 probability of selecting this branch per example, $\epsilon = 0.03$, and $\tau = 3.0$, with the remaining examples using IID corruption. Token losses use the corresponding inverse masking-probability correction.

The frontier variant achieved lower external Phi-4 generative perplexity than the matched IID variant, but substantially lower sliding model-token Distinct-1, Distinct-2, and Distinct-3, accompanied by visibly more immediate token and phrase repetition. This disagreement is precisely why the main experiments retain IID corruption and why generative perplexity is not used as the sole selection criterion. **XXX: Insert the matched run identifiers, checkpoint, generation NFE, token-weighted perplexity, Distinct-1/2/3 values, and representative blinded samples after consolidating the ablation artifacts.**